Appendix 1:

Journal Name: Ecological Monographs

Title: Herbarium specimens reveal complex drivers of herbivory within and among years over more than a century

Authors: Lauren Azevedo-Schmidt[1*], Susan J. Mazer[2] and Emily K. Meineke[1]

**Section S1–**

R code for reproducing Model Set 1.

```
#Purpose: Leaf Level correlation models to understand time (year) and space (lat/long)
  #Create simple correlation models for 1) herbivory ~ year + lat
  # 2) herbivory ~ year + long. Add in DOY into the models if they will allow it.
#Author: LAS
#Date started: 03.19.2025
#Date revised: 08/14/2025
  #Code was revised to reflect updated cleaning code as well as corrections to doy (leapyear) and
adding catalogNumber as random
  #effect to account for pseudoautocorrelation
#R version: 4.3.3.

#Load packages
require(rstan) #need rstan for brms package
require(brms) #bay. models
require(tidyverse)
require(bayesplot)
require(shinystan)

#Function----
#Updated to handle random effects better (THANK GOD!)
tspcorr_brm_models <- function(data, iter = 4000, cores = 4, init = 0,
                               warmup = 2000, chains = 4,
                               control = list(adapt_delta = 0.99, max_treedepth = 15)) {

  # Set seed for reproducibility
  set.seed(888)

  # Create a list to store the models
  models <- list()
```

```
# Check the catalogNumber structure (for information)
catalog_counts <- table(data$catalogNumber)
cat("Leaves per specimen (catalogNumber):\n")
print(summary(catalog_counts))
cat("Total specimens:", length(unique(data$catalogNumber)), "\n")
cat("Total leaves:", nrow(data), "\n\n")

# Get response variable names from the data
freq_vars <- grep("\\.freq$", names(data), value = TRUE)
rich_vars <- grep("\\.rich$", names(data), value = TRUE)
perc_vars <- grep("^leafperc_", names(data), value = TRUE)
chew_vars <- grep("chewvalue", names(data), value = TRUE)

# Combine all response variables
response_vars <- c(freq_vars, rich_vars, perc_vars, chew_vars)

# Function to determine the family based on the response variable name
get_family <- function(response) {
  if (grepl("\\.freq$", response) || grepl("^leafperc_", response) || grepl("chewvalue", response))
{
    return("beta")
  } else if (grepl("\\.rich$", response)) {
    return("zero_inflated_poisson")
  }
}

# Set priors that help with convergence for different families
get_priors <- function(family) {
  if(family == "beta") {
    c(prior(exponential(1), class = sd, group = catalogNumber),
      prior(normal(0, 2.5), class = Intercept),
      prior(normal(0, 1), class = b))
  } else if(family == "zero_inflated_poisson") {
    c(prior(exponential(1), class = sd, group = catalogNumber),
      prior(normal(0, 2), class = Intercept),
      prior(normal(0, 1), class = b),
      prior(beta(1, 1), class = zi))  # Prior for zero-inflation parameter
  } else {
    c(prior(exponential(1), class = sd, group = catalogNumber),
```

```
      prior(normal(0, 2.5), class = Intercept),
      prior(normal(0, 1), class = b))
    }
  }

  # Loop through each response variable
  for (response in response_vars) {
    # Create formulas for different model combinations
    formulas <- list(
      year.lat = paste(response, "~ scale(year.x) + scale(doy.clean) + scale(Lat) +
(1|catalogNumber)"),
      year.long = paste(response, "~ scale(year.x) + scale(doy.clean) + scale(long) +
(1|catalogNumber)")
    )

    # Loop through each formula
    for (model_type in names(formulas)) {
      cat("Fitting", response, "-", model_type, "model...\n")

      formula <- as.formula(formulas[[model_type]])

      # Determine the family
      family <- get_family(response)
      model_priors <- get_priors(family)

      # Fit model with error handling and retries
      model <- NULL
      attempt <- 1
      max_attempts <- 2

      while(is.null(model) && attempt <= max_attempts) {
        if(attempt > 1) {
          cat("  Retry attempt", attempt, "with higher adapt_delta...\n")
          current_control <- list(adapt_delta = 0.995, max_treedepth = 15)
          current_iter <- iter + 1000  # More iterations on retry
          current_warmup <- warmup + 500
        } else {
          current_control <- control
          current_iter <- iter
          current_warmup <- warmup
```

```
}

model <- try({
  brm(formula, data = data, family = family, prior = model_priors,
      iter = current_iter, cores = cores, init = init,
      warmup = current_warmup, chains = chains, control = current_control,
      silent = 2, refresh = 0)  # Reduce output during fitting
}, silent = TRUE)

if(inherits(model, "try-error")) {
  cat("  Error on attempt", attempt, "\n")
  model <- NULL
  attempt <- attempt + 1
} else {
  # Check convergence - robust across brms versions
  rhat_vals <- try({
    # Try newer brms syntax first
    rhat(model, pars = NULL)
  }, silent = TRUE)

  if(inherits(rhat_vals, "try-error")) {
    # Fall back to older syntax or extract from summary
    rhat_vals <- try(summary(model)$fixed[, "Rhat"], silent = TRUE)
  }

  if(!inherits(rhat_vals, "try-error") && length(rhat_vals) > 0) {
    max_rhat <- max(rhat_vals, na.rm = TRUE)
  } else {
    # If all else fails, assume convergence is okay
    max_rhat <- 1.0
    cat("  Could not extract Rhat values\n")
  }

  if(max_rhat > 1.01) {
    cat("  Warning: Max Rhat =", round(max_rhat, 3), "\n")
    if(max_rhat > 1.05 && attempt < max_attempts) {
      cat("  Rhat too high, retrying...\n")
      model <- NULL
      attempt <- attempt + 1
    }
```

```
      } else {
        cat("  Converged successfully (Max Rhat =", round(max_rhat, 3), ")\n")
      }
    }
  }

  if(is.null(model)) {
    cat("  FAILED after", max_attempts, "attempts - skipping", response, model_type, "\n\n")
    next
  }

  # Calculate Bayes R2 with error handling
  r2 <- try(bayes_R2(model), silent = TRUE)
  if(inherits(r2, "try-error")) {
    cat("  Warning: Could not calculate R2 for", response, model_type, "\n")
    r2 <- NA
  }

  # Store the model
  model_name <- paste(response, model_type, sep = "_")
  models[[model_name]] <- model

  # Get the model summary
  model_summary <- summary(model)$fixed

  # Write the model summary to a file
  write.csv(model_summary,
            file = paste0("Script_output/Brms/time_spac_models/l.lvl/young/", model_name,
"_model_summary.csv"))

  # Write full model output (note: this may not work as expected - brms models are complex
objects)
  # Consider saving with saveRDS instead:
  # saveRDS(model, file = paste0("Script_output/Brms/time_spac_models/l.lvl/young/",
model_name, "_model.rds"))
  try({
    write.csv(model,
              file = paste0("Script_output/Brms/time_spac_models/l.lvl/young/", model_name,
"_model.csv"))
  }, silent = TRUE)
```

```
    # Write the R2 information with error handling
    if(!inherits(r2, "try-error") && !any(is.na(r2))) {
      r2_summary <- data.frame(
        Mean_R2 = mean(r2),
        Median_R2 = median(r2),
        Lower_CI = quantile(r2, probs = 0.025),
        Upper_CI = quantile(r2, probs = 0.975),
        row.names = "Bayes_R2"
      )

      write.csv(r2_summary,
                file = paste0("Script_output/Brms/time_spac_models/l.lvl/young/", model_name,
"_model_summary_R2.csv"))
    } else {
      cat("  Skipping R2 output due to calculation error\n")
    }

    cat("  Saved:", model_name, "\n\n")
  }
 }

 cat("Model fitting complete!\n")
 cat("Successfully fitted", length(models), "out of", length(response_vars) * 2, "attempted
models.\n")

 return(models)
}



#Load leaf level data----
oldleaves <- read_csv("Cleaned_data/cleaned_reclass.old.leaves.csv")
youngleaves <- read_csv("Cleaned_data/cleaned_reclass.young.leaves.csv")

#recombine old and young datasets to make a full dataset that is the same number of leaves as
the seperated datasets
leafdata2 <- youngleaves %>%
 rbind(oldleaves)
```

```
#Subset data for time/space models
oldleaves4 <- oldleaves %>%
  select(c(ends_with(".freq"), ends_with(".rich"), starts_with("leafperc_"), "chewvalue","year.x",
"doy.clean", "Lat", "long","catalogNumber"))

youngleaves4 <- youngleaves %>%
  select(c(ends_with(".freq"), ends_with(".rich"), starts_with("leafperc_"), "chewvalue","year.x",
"doy.clean", "Lat", "long","catalogNumber"))


# Using function----
tspcorr_oldmodels <- tspcorr_brm_models(oldleaves4)
tspcorr_youngmodels <- tspcorr_brm_models(youngleaves4)




#Old cleaning code that is now redudant:
#Below is the old not reclassified leaf_rel_ages
#QL_leafdata <- read_csv("Cleaned_data/QL_leaflevel.df.csv") #LEAF LEVEL DATA ONLY
#clmdata <- read_csv("Cleaned_data/climate.data.csv")

# oldleaves2 <- oldleaves%>%
#   select(!c("perc_area_PS", "perc_area_ovi")) %>%
#   mutate_at(c("leafperc_area_chew", "leafperc_area_mine", "leafperc_area_gall",
#               "leaftotal.freq", "leafspec.freq", "leafgall.freq",
#               "leafmine.freq"), as.numeric) %>% #R is importing these as characters which isn't
correct so updating them here
#   mutate(leaf_rel_age = as.factor(leaf_rel_age)) %>%
#   select(c("MAT":"CMI_12", "doy.clean","catalogNumber",
"leaf_rel_age","year.x","leafperc_area_chew", "leafperc_area_mine", "leafperc_area_gall",
#           "leaftotal.freq", "leafspec.freq", "leafgall.freq",
#           "leafmine.freq", "leaftotal.rich", "leafspec.rich", "leafgall.rich", "leafmine.rich", "lat",
"lon")) %>%
#   mutate(across(starts_with("leafperc_area_"), ~ . / 100)) %>% #converting percent leaf area
measurements to proportions (0:1)
#   mutate(leafgen.freq = leaftotal.freq - leafspec.freq, #creating generalist columns for frequency
and richness by supstracting specialist from total
#          leafgen.rich = leaftotal.rich - leafspec.rich)
```

```
#
# # #converting all 0 = 0.0001 and 1 = 0.9999
# oldleaves2$leaftotal.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leaftotal.freq))
# oldleaves2$leafgen.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leafgen.freq))
# oldleaves2$leafspec.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leafspec.freq))
# oldleaves2$leafmine.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leafmine.freq))
# oldleaves2$leafgall.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leafgall.freq))
# oldleaves2$leafperc_area_chew <- pmax(0.0001, pmin(0.9999,
oldleaves2$leafperc_area_chew))
# oldleaves2$leafperc_area_mine <- pmax(0.0001, pmin(0.9999,
oldleaves2$leafperc_area_mine))
# oldleaves2$leafperc_area_gall <- pmax(0.0001, pmin(0.9999, oldleaves2$leafperc_area_gall))
#
# youngleaves2 <- youngleaves%>%
#   select(!c("perc_area_PS", "perc_area_ovi")) %>%
#   mutate_at(c("leafperc_area_chew", "leafperc_area_mine", "leafperc_area_gall",
#               "leaftotal.freq", "leafspec.freq", "leafgall.freq",
#               "leafmine.freq"), as.numeric) %>% #R is importing these as characters which isn't
correct so updating them here
#   mutate(leaf_rel_age = as.factor(leaf_rel_age)) %>%
#   select(c("MAT":"CMI_12","doy.clean","catalogNumber",
"leaf_rel_age","year.x","leafperc_area_chew", "leafperc_area_mine", "leafperc_area_gall",
#           "leaftotal.freq", "leafspec.freq", "leafgall.freq",
#           "leafmine.freq", "leaftotal.rich", "leafspec.rich", "leafgall.rich", "leafmine.rich", "lat",
"lon")) %>%
#   mutate(across(starts_with("leafperc_area_"), ~ . / 100)) %>% #converting percent leaf area
measurements to proportions (0:1)
#   mutate(leafgen.freq = leaftotal.freq - leafspec.freq, #creating generalist columns for frequency
and richness by supstracting specialist from total
#          leafgen.rich = leaftotal.rich - leafspec.rich)
#
# # #converting all 0 = 0.0001 and 1 = 0.9999
# youngleaves2$leaftotal.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leaftotal.freq))
# youngleaves2$leafgen.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leafgen.freq))
# youngleaves2$leafspec.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leafspec.freq))
# youngleaves2$leafmine.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leafmine.freq))
# youngleaves2$leafgall.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leafgall.freq))
# youngleaves2$leafperc_area_chew <- pmax(0.0001, pmin(0.9999,
youngleaves2$leafperc_area_chew))
```

```
# youngleaves2$leafperc_area_mine <- pmax(0.0001, pmin(0.9999,
youngleaves2$leafperc_area_mine))
# youngleaves2$leafperc_area_gall <- pmax(0.0001, pmin(0.9999,
youngleaves2$leafperc_area_gall))
#
# #creating a new column of data that is 0:1 for chewing damage to then run models on
# oldleaves3 <- oldleaves2
# oldleaves3$chewbinary <- oldleaves3$leafperc_area_chew
# oldleaves3 <- oldleaves3 %>%
#   mutate(value = if_else(chewbinary < 0.0001, 0, 1)) %>%
#   rename(chewvalue = value) %>%
#   select(!chewbinary)
# oldleaves3$chewvalue <- pmax(0.0001, pmin(0.9999, oldleaves3$chewvalue))
# oldleaves3$leafperc_area_chew <- round(oldleaves3$leafperc_area_chew, 4) #rounding
percent area damaged columns to only have 4 decimal places
# oldleaves3$leafperc_area_gall <- round(oldleaves3$leafperc_area_gall, 4) #this will make all
columns standarized
# oldleaves3$leafperc_area_mine <- round(oldleaves3$leafperc_area_mine, 4)
#
# #removing columns with incorrect climate data (i.e., -9999.0)
# oldleaves3 <- oldleaves3%>%
#   select(where(~if_else(any(. == -9999.0, na.rm = TRUE), FALSE, TRUE)))
#
# youngleaves3 <- youngleaves2
# youngleaves3$chewbinary <- youngleaves3$leafperc_area_chew
# youngleaves3 <- youngleaves3 %>%
#   mutate(value = if_else(chewbinary < 0.0001, 0, 1)) %>%
#   rename(chewvalue = value) %>%
#   select(!chewbinary)
# youngleaves3$chewvalue <- pmax(0.0001, pmin(0.9999, youngleaves3$chewvalue))
# youngleaves3$leafperc_area_chew <- round(youngleaves3$leafperc_area_chew, 4)
# youngleaves3$leafperc_area_gall <- round(youngleaves3$leafperc_area_gall, 4)
# youngleaves3$leafperc_area_mine <- round(youngleaves3$leafperc_area_mine, 4)
#
# youngleaves3<- youngleaves3%>%
#   select(where(~if_else(any(. == -9999.0, na.rm = TRUE), FALSE, TRUE)))

#OLD function----
# tspcorr_brm_models <- function(data, iter = 4000, cores = 2, init = 0,
#                                warmup = 2000, chains = 4,
```

```
#                              control = list(adapt_delta = 0.95)) {
#
#   # Set seed for reproducibility
#   set.seed(888)
#
#   # Create a list to store the models
#   models <- list()
#
#   # Get response variable names from the data
#   freq_vars <- grep("\\.freq$", names(data), value = TRUE)
#   rich_vars <- grep("\\.rich$", names(data), value = TRUE)
#   perc_vars <- grep("^leafperc_", names(data), value = TRUE)
#   chew_vars <- grep("chewvalue", names(data), value = TRUE)  # Added chew value
#
#   # Combine all response variables
#   response_vars <- c(freq_vars, rich_vars, perc_vars, chew_vars)
#
#   # Function to determine the family based on the response variable name
#   get_family <- function(response) {
#     if (grepl("\\.freq$", response) || grepl("^leafperc_", response) || grepl("chewvalue",
response)) {
#       return("beta")
#     } else if (grepl("\\.rich$", response)) {
#       return("zero_inflated_poisson")
#     }
#   }
#
#   # Loop through each response variable
#   for (response in response_vars) {
#     # Create formulas for different model combinations
#     formulas <- list(
#       year.lat = paste(response, "~ scale(year.x) + scale(doy.clean) + scale(Lat) +
(1|catalogNumber)"), #might have to remove this if DOY breaks the model
#       #doy = paste(response, "~ scale(doy.clean)"),
#       #lat = paste(response, "~scale(lat)"),
#       year.long = paste(response, "~ scale(year.x) + scale(doy.clean) + scale(long)+
(1|catalogNumber)")
#       #interaction = paste(response, "~ scale(year.x) * scale(doy.clean)")
#     )
#
```

```
#     # Loop through each formula
#     for (model_type in names(formulas)) {
#       formula <- as.formula(formulas[[model_type]])
#
#       # Determine the family
#       family <- get_family(response)
#
#       # Fit the model
#       model <- brm(formula, data = data, family = family,
#                    iter = iter, cores = cores, init = init,
#                    warmup = warmup, chains = chains, control = control)
#
#       # Calculate Bayes R2
#       r2 <- bayes_R2(model)
#
#       # Store the model
#       model_name <- paste(response, model_type, sep = "_")
#       models[[model_name]] <- model
#
#       # Get the model summary
#       model_summary <- summary(model)$fixed
#
#       # Write the model summary to a file
#       write.csv(model_summary,
#                 file = paste0("Script_output/Brms/time_spac_models/l.lvl/young/", model_name,
"_model_summary.csv"))
#
#       # Write full model output
#       write.csv(model,
#                 file = paste0("Script_output/Brms/time_spac_models/l.lvl/young/", model_name,
"_model.csv"))
#
#       # Write the R2 information in the same directory but with the summary in the filename
#       r2_summary <- data.frame(
#         Mean_R2 = mean(r2),
#         Median_R2 = median(r2),
#         Lower_CI = quantile(r2, probs = 0.025),
#         Upper_CI = quantile(r2, probs = 0.975),
#         row.names = "Bayes_R2"  # Set an explicit row name
#       )
```

```
#
#       write.csv(r2_summary,
#                 file = paste0("Script_output/Brms/time_spac_models/l.lvl/young/", model_name,
"_model_summary_R2.csv"))
#     }
#   }
#
#   return(models)
# }
```

**Section S2 –**

R code for reproducing Model Set 2.

```
#Purpose: Run LEAF LEVEL simple models
#R version: 4.3.3.
#Author: LAS
#Date started: 08.28.2024
  #Date updated: 08/18/2025
    #This code has been updated to handle random effects to account for pseudoreplication
(catalogNumber)
    #It includes all updates reflected in notes on "STEP3D_leaf.level.models.function"


#load packages
require(rstan) #need rstan for brms package
require(brms) #bay. models
require(tidyverse)
require(bayesplot)
require(shinystan)

#Function
simp_brm_models <- function(data, iter = 4000, cores = 4, init = 0,
                            warmup = 2000, chains = 4,
                            control = list(adapt_delta = 0.99, max_treedepth = 15)) {

  # Set seed for reproducibility
  set.seed(888)
```

```
# Create a list to store the models
models <- list()

# Check the catalogNumber structure (for information)
catalog_counts <- table(data$catalogNumber)
cat("Leaves per specimen (catalogNumber):\n")
print(summary(catalog_counts))
cat("Total specimens:", length(unique(data$catalogNumber)), "\n")
cat("Total leaves:", nrow(data), "\n\n")

# Get response variable names from the data
freq_vars <- grep("\\.freq$", names(data), value = TRUE)
rich_vars <- grep("\\.rich$", names(data), value = TRUE)
perc_vars <- grep("^leafperc_", names(data), value = TRUE)
chew_vars <- grep("chewvalue", names(data), value = TRUE)

# Combine all response variables
response_vars <- c(freq_vars, rich_vars, perc_vars, chew_vars)

# Function to determine the family based on the response variable name
get_family <- function(response) {
  if (grepl("\\.freq$", response) || grepl("^leafperc_", response) || grepl("chewvalue", response))
{
    return("beta")
  } else if (grepl("\\.rich$", response)) {
    return("zero_inflated_poisson")
  }
}

# Set priors that help with convergence for different families
get_priors <- function(family) {
  if(family == "beta") {
    c(prior(exponential(1), class = sd, group = catalogNumber),
      prior(normal(0, 2.5), class = Intercept),
      prior(normal(0, 1), class = b))
  } else if(family == "zero_inflated_poisson") {
    c(prior(exponential(1), class = sd, group = catalogNumber),
      prior(normal(0, 2), class = Intercept),
      prior(normal(0, 1), class = b),
```

```
      prior(beta(1, 1), class = zi))  # Prior for zero-inflation parameter
    } else {
      c(prior(exponential(1), class = sd, group = catalogNumber),
        prior(normal(0, 2.5), class = Intercept),
        prior(normal(0, 1), class = b))
    }
  }

  # Loop through each response variable
  for (response in response_vars) {
    # Create formulas for different model combinations
    formulas <- list(
      year_doy.int = paste(response, "~ scale(year.x) + scale(doy.clean)+ scale(year.x) *
scale(doy.clean) +(1|catalogNumber)")
      #doy = paste(response, "~ scale(doy.clean) + (1|catalogNumber)"),
      #interaction = paste(response, "~ scale(year.x) * scale(doy.clean) + (1|catalogNumber)")
    )

    # Loop through each formula
    for (model_type in names(formulas)) {
      cat("Fitting", response, "-", model_type, "model...\n")

      formula <- as.formula(formulas[[model_type]])

      # Determine the family
      family <- get_family(response)
      model_priors <- get_priors(family)

      # Fit model with error handling and retries
      model <- NULL
      attempt <- 1
      max_attempts <- 2

      while(is.null(model) && attempt <= max_attempts) {
        if(attempt > 1) {
          cat("  Retry attempt", attempt, "with higher adapt_delta...\n")
          current_control <- list(adapt_delta = 0.995, max_treedepth = 15)
          current_iter <- iter + 1000  # More iterations on retry
          current_warmup <- warmup + 500
        } else {
```

```
  current_control <- control
  current_iter <- iter
  current_warmup <- warmup
}

model <- try({
  brm(formula, data = data, family = family, prior = model_priors,
      iter = current_iter, cores = cores, init = init,
      warmup = current_warmup, chains = chains, control = current_control,
      silent = 2, refresh = 0)  # Reduce output during fitting
}, silent = TRUE)

if(inherits(model, "try-error")) {
  cat("  Error on attempt", attempt, "\n")
  model <- NULL
  attempt <- attempt + 1
} else {
  # Check convergence - robust across brms versions
  rhat_vals <- try({
    # Try newer brms syntax first
    rhat(model, pars = NULL)
  }, silent = TRUE)

  if(inherits(rhat_vals, "try-error")) {
    # Fall back to older syntax or extract from summary
    rhat_vals <- try(summary(model)$fixed[, "Rhat"], silent = TRUE)
  }

  if(!inherits(rhat_vals, "try-error") && length(rhat_vals) > 0) {
    max_rhat <- max(rhat_vals, na.rm = TRUE)
  } else {
    # If all else fails, assume convergence is okay
    max_rhat <- 1.0
    cat("  Could not extract Rhat values\n")
  }

  if(max_rhat > 1.01) {
    cat("  Warning: Max Rhat =", round(max_rhat, 3), "\n")
    if(max_rhat > 1.05 && attempt < max_attempts) {
      cat("  Rhat too high, retrying...\n")
```

```
          model <- NULL
          attempt <- attempt + 1
        }
      } else {
        cat("  Converged successfully (Max Rhat =", round(max_rhat, 3), ")\n")
      }
    }
  }

  if(is.null(model)) {
    cat("  FAILED after", max_attempts, "attempts - skipping", response, model_type, "\n\n")
    next
  }

  # Calculate Bayes R2 with error handling
  r2 <- try(bayes_R2(model), silent = TRUE)
  if(inherits(r2, "try-error")) {
    cat("  Warning: Could not calculate R2 for", response, model_type, "\n")
    r2 <- NA
  }

  # Store the model
  model_name <- paste(response, model_type, sep = "_")
  models[[model_name]] <- model

  # Write full model output (consider using saveRDS instead)
  try({
    write.csv(model,
              file = paste0("Script_output/Brms/simp_models/l.lvl/young/", model_name,
"_model.csv"))
  }, silent = TRUE)

  # Write the model summary
  model_summary <- summary(model)$fixed
  write.csv(model_summary,
            file = paste0("Script_output/Brms/simp_models/l.lvl/young/", model_name,
"_model_summary.csv"))

  # Write R2 files with error handling
  if(!inherits(r2, "try-error") && !any(is.na(r2))) {
```

```
      write.csv(as.data.frame(r2),
                paste0("Script_output/Brms/simp_models/l.lvl/young/", model_name,
"_bayes_R2.csv"))
     } else {
       cat("  Skipping R2 output due to calculation error\n")
     }

     cat("  Saved:", model_name, "\n\n")
   }
  }

  cat("Model fitting complete!\n")
  cat("Successfully fitted", length(models), "out of", length(response_vars), "attempted
models.\n")

  return(models)
}


#load data and subset----
oldleaves <- read_csv("Cleaned_data/cleaned_reclass.old.leaves.csv")
youngleaves <- read_csv("Cleaned_data/cleaned_reclass.young.leaves.csv")

#recombine old and young datasets to make a full dataset that is the same number of leaves as
the seperated datasets
# leafdata2 <- youngleaves %>%
#   rbind(oldleaves)

#Subset data for simple models
oldleaves4 <- oldleaves %>%
  select(c(ends_with(".freq"), ends_with(".rich"), starts_with("leafperc_"), "chewvalue","year.x",
"doy.clean", "catalogNumber"))

youngleaves4 <- youngleaves %>%
  select(c(ends_with(".freq"), ends_with(".rich"), starts_with("leafperc_"), "chewvalue","year.x",
"doy.clean","catalogNumber"))

# Usage of function----
#simple year and doy models below:
simp_oldmodels <- simp_brm_models(oldleaves4)
```

```
simp_youngmodels <- simp_brm_models(youngleaves4)
```

**Section S3–**
R code for reproducing Model Set 3.

```
#Purpose: LEAF level models with LEAF level PCA importance
  #This uses a function to import the PCA importance files and run through each brm()
  #for each unique response and predictor variable. Then, it writes a .csv file for the summary
  #output and the full model output.
#R version: 4.3.3.
#Aurhors: LAS
#Date started: 08.28.2024
#Date last revised: 08/14/2025
  #Code has been edited to simplify cleaning across all scripts. Full cleaning is now done in the
clean.combine_data script
  #Additionally, models were re-run with corrected climate data. PCAs were also re-run prior to
this step.

#Date last revised: 02/24/2025
    #Added in rounding of all herbivory columns to 4 decimals
    #Model outputs were all re-run on this date and are update to date as of now. SHOULD be the
last time we need to rerun them.
#Previous update notes
    #11/13/2024
    #Script was revised to add the "simp_brm_models" function that looks at the herbivory
response variables by year and doy but
    #no climate variables. This was we can verify our results from models that hold year and doy
constant.

#load packages
require(rstan) #need rstan for brms package
require(brms) #bay. models
require(tidyverse)
require(bayesplot)
require(shinystan)

#climate function----
climate_brm_models <- function(data_file, response_data, additional_predictors = NULL,
                               iter = 4000, cores = 4, init = 0,
```

```
                    warmup = 2000, chains = 4,
                    control = list(adapt_delta = 0.99, max_treedepth = 15)) {

  # Set seed for reproducibility
  set.seed(888)

  # Read the CSV file
  data <- read.csv(data_file)

  # Check the catalogNumber structure in response_data (for information)
  if("catalogNumber" %in% names(response_data)) {
    catalog_counts <- table(response_data$catalogNumber)
    cat("Leaves per specimen (catalogNumber) in response_data:\n")
    print(summary(catalog_counts))
    cat("Total specimens:", length(unique(response_data$catalogNumber)), "\n")
    cat("Total leaves:", nrow(response_data), "\n\n")
  }

  # Create a list to store the models
  models <- list()

  # Function to determine the family based on the response variable name
  get_family <- function(response) {
    if (grepl("^leaf.*_area_", response) || grepl("\\.freq$", response) || grepl("chewvalue",
response)) {
      return("beta")
    } else if (grepl("\\.rich$", response)) {
      return("zero_inflated_poisson")
    } else {
      stop(paste("Unknown family for response variable:", response))
    }
  }

  # Set priors that help with convergence for different families
  get_priors <- function(family) {
    if(family == "beta") {
      c(prior(exponential(1), class = sd, group = catalogNumber),
        prior(normal(0, 2.5), class = Intercept),
        prior(normal(0, 1), class = b))
    } else if(family == "zero_inflated_poisson") {
```

```
    c(prior(exponential(1), class = sd, group = catalogNumber),
      prior(normal(0, 2), class = Intercept),
      prior(normal(0, 1), class = b),
      prior(beta(1, 1), class = zi))  # Prior for zero-inflation parameter
  } else {
    c(prior(exponential(1), class = sd, group = catalogNumber),
      prior(normal(0, 2.5), class = Intercept),
      prior(normal(0, 1), class = b))
  }
}

# Function to check convergence robustly across brms versions
check_convergence <- function(model) {
  # Try different methods to extract Rhat values
  rhat_vals <- try({
    # Try newer brms syntax first
    rhat(model, pars = NULL)
  }, silent = TRUE)

  if(inherits(rhat_vals, "try-error")) {
    # Fall back to extracting from summary
    rhat_vals <- try({
      model_summary <- summary(model)
      model_summary$fixed[, "Rhat"]
    }, silent = TRUE)
  }

  if(!inherits(rhat_vals, "try-error") && length(rhat_vals) > 0) {
    max_rhat <- max(rhat_vals, na.rm = TRUE)
  } else {
    # If all else fails, assume convergence is okay but warn
    max_rhat <- 1.0
    warning("Could not extract Rhat values for convergence check")
  }

  return(max_rhat)
}

# Get unique response variables
response_vars <- unique(data$Response)
```

```
  cat("Found", length(response_vars), "response variables:", paste(response_vars, collapse = ", "),
"\n\n")

  # Define model types
  model_types <- list(
  year_doy = function(response) paste(response, "~ scale(year.x) + scale(doy.clean) +
(1|catalogNumber)")
)
  #uncomment code below if year and doy are wanted to be separate.
  # model_types <- list(
  #   year = function(response) paste(response, "~ scale(year.x) + (1|catalogNumber)"),
  #   doy = function(response) paste(response, "~ scale(doy.clean) + (1|catalogNumber)")
  # )

  # Count total models to fit
  total_models <- 0
  for (response in response_vars) {
    predictors <- data %>%
      filter(Response == response) %>%
      pull(Variable)
    all_predictors <- unique(c(predictors, additional_predictors))
    total_models <- total_models + (length(all_predictors) * length(model_types))
  }

  cat("Planning to fit", total_models, "models total\n\n")

  model_counter <- 0
  successful_models <- 0

  # Loop through each response variable
  for (response in response_vars) {
    cat("=== Processing response variable:", response, "===\n")

    # Get predictor variables for the current response
    predictors <- data %>%
      filter(Response == response) %>%
      pull(Variable)

    # Combine with additional predictors if provided
    all_predictors <- unique(c(predictors, additional_predictors))
```

```
cat("Predictors for", response, ":", paste(all_predictors, collapse = ", "), "\n\n")

# Loop through each predictor for the current response
for (predictor in all_predictors) {
  # Loop through each model type
  for (model_type_name in names(model_types)) {
    model_counter <- model_counter + 1
    cat("Model", model_counter, "of", total_models, ":")
    cat("Fitting", response, "-", predictor, "-", model_type_name, "...\n")

    # Create the formula using the appropriate model type function
    base_formula <- model_types[[model_type_name]](response)

    # Add the predictor to the formula if it's not already part of the model type
    if (!grepl(predictor, base_formula, fixed = TRUE)) {
      formula_str <- paste(base_formula, "+", paste0("scale(", predictor, ")"))
    } else {
      formula_str <- base_formula
    }

    formula <- as.formula(formula_str)
    cat("  Formula:", deparse(formula), "\n")

    # Determine the family and priors
    family <- get_family(response)
    model_priors <- get_priors(family)

    # Fit model with error handling and retries
    model <- NULL
    attempt <- 1
    max_attempts <- 2

    while(is.null(model) && attempt <= max_attempts) {
      if(attempt > 1) {
        cat("  Retry attempt", attempt, "with higher adapt_delta...\n")
        current_control <- list(adapt_delta = 0.995, max_treedepth = 15)
        current_iter <- iter + 1000
        current_warmup <- warmup + 500
      } else {
        current_control <- control
```

```
    current_iter <- iter
    current_warmup <- warmup
  }

  model <- try({
    brm(formula, data = response_data, family = family, prior = model_priors,
        iter = current_iter, cores = cores, init = init,
        warmup = current_warmup, chains = chains, control = current_control,
        silent = 2, refresh = 0)
  }, silent = TRUE)

  if(inherits(model, "try-error")) {
    cat("  Error on attempt", attempt, "\n")
    model <- NULL
    attempt <- attempt + 1
  } else {
    # Check convergence
    max_rhat <- check_convergence(model)

    if(max_rhat > 1.01) {
      cat("  Warning: Max Rhat =", round(max_rhat, 3), "\n")
      if(max_rhat > 1.05 && attempt < max_attempts) {
        cat("  Rhat too high, retrying...\n")
        model <- NULL
        attempt <- attempt + 1
      }
    } else {
      cat("  Converged successfully (Max Rhat =", round(max_rhat, 3), ")\n")
    }
  }
}

if(is.null(model)) {
  cat("  FAILED after", max_attempts, "attempts - skipping\n\n")
  next
}

# Calculate Bayes R2 with error handling
r2 <- try(bayes_R2(model), silent = TRUE)
if(inherits(r2, "try-error")) {
```

```
      cat("  Warning: Could not calculate R2\n")
      r2 <- NA
    }

    # Store the model
    model_name <- paste(response, predictor, model_type_name, sep = "_")
    models[[model_name]] <- model
    successful_models <- successful_models + 1

    # Create output directory if it doesn't exist
    output_dir <- paste0("Script_output/Brms/climate_models/l.lvl/young/", model_type_name,
"/")
    dir.create(output_dir, recursive = TRUE, showWarnings = FALSE)

    # Write outputs with error handling
    try({
      # Note: writing brmsfit objects to CSV may not work well - consider saveRDS
      # saveRDS(model, paste0(output_dir, model_name, "_model.rds"))
      write.csv(model, paste0(output_dir, model_name, "_model.csv"))
    }, silent = TRUE)

    # Write the model summary
    try({
      write.csv(summary(model)$fixed,
              paste0(output_dir, model_name, "_model.summary.csv"))
    }, silent = TRUE)

    # Write the Bayes R2 if available
    if(!inherits(r2, "try-error") && !any(is.na(r2))) {
      try({
        write.csv(as.data.frame(r2),
                paste0(output_dir, model_name, "_bayes_R2.csv"))
      }, silent = TRUE)
    }

    cat("  Saved:", model_name, "\n\n")
  }
}
cat("=== Completed response variable:", response, "===\n\n")
}
```

```
  cat("=== MODEL FITTING COMPLETE ===\n")
  cat("Successfully fitted", successful_models, "out of", total_models, "attempted models.\n")
  cat("Success rate:", round(100 * successful_models / total_models, 1), "%\n")

  return(models)
}


#load data and subset----
oldleaves <- read_csv("Cleaned_data/cleaned_reclass.old.leaves.csv")
youngleaves <- read_csv("Cleaned_data/cleaned_reclass.young.leaves.csv")

#run models----
oldYmodels <- climate_brm_models("Script_output/PCA/l.lvl/old/Ltop5oldY.csv", oldleaves)
youngYmodels <- climate_brm_models("Script_output/PCA/l.lvl/young/Ltop5youngY.csv",
youngleaves)



#################SCRATCH PAPER AND OLD
CODE################################

#scratch paper to fix when errors occur
# Ltop5Y <- read.csv("Script_output/PCA/Ltop5Y.csv")
# cb <- Ltop5Y %>%
#     filter(Response == "chewbinary")
# write.csv(cb, file = "Script_output/PCA/cb.csv")


#Old cleaning code that is now redundant:
# oldleaves2 <- oldleaves%>%
#   select(!c("perc_area_PS", "perc_area_ovi")) %>%
#   mutate_at(c("leafperc_area_chew", "leafperc_area_mine", "leafperc_area_gall",
#               "leaftotal.freq", "leafspec.freq", "leafgall.freq",
#               "leafmine.freq"), as.numeric) %>% #R is importing these as characters which isn't
correct so updating them here
#   mutate(leaf_rel_age = as.factor(leaf_rel_age)) %>%
#   select(c("MAT":"CMI_12", "doy.clean","catalogNumber",
"leaf_rel_age","year.x","leafperc_area_chew", "leafperc_area_mine", "leafperc_area_gall",
```

```
#           "leaftotal.freq", "leafspec.freq", "leafgall.freq",
#           "leafmine.freq", "leaftotal.rich", "leafspec.rich", "leafgall.rich", "leafmine.rich")) %>%
#   mutate(across(starts_with("leafperc_area_"), ~ . / 100)) %>% #converting percent leaf area
measurements to proportions (0:1)
#   mutate(leafgen.freq = leaftotal.freq - leafspec.freq, #creating generalist columns for frequency
and richness by supstracting specialist from total
#         leafgen.rich = leaftotal.rich - leafspec.rich)
#
# # #converting all 0 = 0.0001 and 1 = 0.9999
# oldleaves2$leaftotal.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leaftotal.freq))
# oldleaves2$leafgen.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leafgen.freq))
# oldleaves2$leafspec.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leafspec.freq))
# oldleaves2$leafmine.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leafmine.freq))
# oldleaves2$leafgall.freq <- pmax(0.0001, pmin(0.9999, oldleaves2$leafgall.freq))
# oldleaves2$leafperc_area_chew <- pmax(0.0001, pmin(0.9999,
oldleaves2$leafperc_area_chew))
# oldleaves2$leafperc_area_mine <- pmax(0.0001, pmin(0.9999,
oldleaves2$leafperc_area_mine))
# oldleaves2$leafperc_area_gall <- pmax(0.0001, pmin(0.9999, oldleaves2$leafperc_area_gall))
#
# youngleaves2 <- youngleaves%>%
#   select(!c("perc_area_PS", "perc_area_ovi")) %>%
#   mutate_at(c("leafperc_area_chew", "leafperc_area_mine", "leafperc_area_gall",
#             "leaftotal.freq", "leafspec.freq", "leafgall.freq",
#             "leafmine.freq"), as.numeric) %>% #R is importing these as characters which isn't
correct so updating them here
#   mutate(leaf_rel_age = as.factor(leaf_rel_age)) %>%
#   select(c("MAT":"CMI_12","doy.clean","catalogNumber",
"leaf_rel_age","year.x","leafperc_area_chew", "leafperc_area_mine", "leafperc_area_gall",
#           "leaftotal.freq", "leafspec.freq", "leafgall.freq",
#           "leafmine.freq", "leaftotal.rich", "leafspec.rich", "leafgall.rich", "leafmine.rich")) %>%
#   mutate(across(starts_with("leafperc_area_"), ~ . / 100)) %>% #converting percent leaf area
measurements to proportions (0:1)
#   mutate(leafgen.freq = leaftotal.freq - leafspec.freq, #creating generalist columns for frequency
and richness by supstracting specialist from total
#         leafgen.rich = leaftotal.rich - leafspec.rich)
#
# # #converting all 0 = 0.0001 and 1 = 0.9999
# youngleaves2$leaftotal.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leaftotal.freq))
# youngleaves2$leafgen.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leafgen.freq))
```

```
# youngleaves2$leafspec.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leafspec.freq))
# youngleaves2$leafmine.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leafmine.freq))
# youngleaves2$leafgall.freq <- pmax(0.0001, pmin(0.9999, youngleaves2$leafgall.freq))
# youngleaves2$leafperc_area_chew <- pmax(0.0001, pmin(0.9999,
youngleaves2$leafperc_area_chew))
# youngleaves2$leafperc_area_mine <- pmax(0.0001, pmin(0.9999,
youngleaves2$leafperc_area_mine))
# youngleaves2$leafperc_area_gall <- pmax(0.0001, pmin(0.9999,
youngleaves2$leafperc_area_gall))
#
# #creating a new column of data that is 0:1 for chewing damage to then run models on
# oldleaves3 <- oldleaves2
# oldleaves3$chewbinary <- oldleaves3$leafperc_area_chew
# oldleaves3 <- oldleaves3 %>%
#   mutate(value = if_else(chewbinary < 0.0001, 0, 1)) %>%
#   rename(chewvalue = value) %>%
#   select(!chewbinary)
# oldleaves3$chewvalue <- pmax(0.0001, pmin(0.9999, oldleaves3$chewvalue))
# oldleaves3$leafperc_area_chew <- round(oldleaves3$leafperc_area_chew, 4) #rounding
percent area damaged columns to only have 4 decimal places
# oldleaves3$leafperc_area_gall <- round(oldleaves3$leafperc_area_gall, 4) #this will make all
columns standarized
# oldleaves3$leafperc_area_mine <- round(oldleaves3$leafperc_area_mine, 4)
#
# #removing columns with incorrect climate data (i.e., -9999.0)
# oldleaves3 <- oldleaves3%>%
#   select(where(~if_else(any(. == -9999.0, na.rm = TRUE), FALSE, TRUE)))
#
# youngleaves3 <- youngleaves2
# youngleaves3$chewbinary <- youngleaves3$leafperc_area_chew
# youngleaves3 <- youngleaves3 %>%
#   mutate(value = if_else(chewbinary < 0.0001, 0, 1)) %>%
#   rename(chewvalue = value) %>%
#   select(!chewbinary)
# youngleaves3$chewvalue <- pmax(0.0001, pmin(0.9999, youngleaves3$chewvalue))
# youngleaves3$leafperc_area_chew <- round(youngleaves3$leafperc_area_chew, 4)
# youngleaves3$leafperc_area_gall <- round(youngleaves3$leafperc_area_gall, 4)
# youngleaves3$leafperc_area_mine <- round(youngleaves3$leafperc_area_mine, 4)
#
# youngleaves3<- youngleaves3%>%
```

```
#   select(where(~if_else(any(. == -9999.0, na.rm = TRUE), FALSE, TRUE)))




#Leaf Level brms() function
#functions----
#climate variable models
# create_brm_models <- function(data_file, response_data, additional_predictors = NULL,
#                               iter = 2000, cores = 2, init = 0,
#                               warmup = 200, chains = 2,
#                               control = list(adapt_delta = 0.95)) {
#
#   #set seed
#   set.seed(888)
#
#   # Read the CSV file
#   data <- read.csv(data_file)
#
#   # Create a list to store the models
#   models <- list()
#
#   # Function to determine the family based on the response variable name
#   get_family <- function(response) {
#     if (grepl("^leaf.*_area_", response) || grepl("\\.freq$", response) ||  grepl("chewvalue",
response)) {
#       return("beta")
#     } else if (grepl("\\.rich$", response)) {
#       return("zero_inflated_poisson")
#     } else {
#       stop(paste("Unknown family for response variable:", response))
#     }
#   }
#
#   # Get unique response variables
#   response_vars <- unique(data$Response)
#
#   # Define model types
#   model_types <- list(
#     year = function(response) paste(response, "~ scale(year.x)+ (1|catalogNumber)"),
```

```
#     doy = function(response) paste(response, "~ scale(doy.clean)+ (1|catalogNumber)") #add
comma back if the interaction is wanted
#     #interaction = function(response) paste(response, "~ scale(year.x) * scale(doy.clean)")
#   )
#
#   # Loop through each response variable
#   for (response in response_vars) {
#     # Get predictor variables for the current response
#     predictors <- data %>%
#       filter(Response == response) %>%
#       pull(Variable)
#
#     # Combine with additional predictors if provided
#     all_predictors <- unique(c(predictors, additional_predictors))
#
#     # Loop through each predictor for the current response
#     for (predictor in all_predictors) {
#       # Loop through each model type
#       for (model_type_name in names(model_types)) {
#         # Create the formula using the appropriate model type function
#         base_formula <- model_types[[model_type_name]](response)
#
#         # Add the predictor to the formula if it's not already part of the model type
#         if (!grepl(predictor, base_formula, fixed = TRUE)) {
#           formula_str <- paste(base_formula, "+", paste0("scale(", predictor, ")"))
#         } else {
#           formula_str <- base_formula
#         }
#
#         formula <- as.formula(formula_str)
#
#         # Determine the family
#         family <- get_family(response)
#
#         # Fit the model
#         model <- brm(formula, data = response_data, family = family,
#                      iter = iter, cores = cores, init = init, warmup = warmup,
#                      chains = chains, control = control)
#
#         # Calculate Bayes R2
```

```
#        r2 <- bayes_R2(model)
#
#        # Store the model
#        model_name <- paste(response, predictor, model_type_name, sep = "_")
#        models[[model_name]] <- model
#
#        # Create output directory if it doesn't exist
#        output_dir <- paste0("Script_output/Brms/climate_models/l.lvl/old/", model_type_name,
"/") #switch directory between old/young runs
#        dir.create(output_dir, recursive = TRUE, showWarnings = FALSE)
#
#        # Write full model output
#        write.csv(model, paste0(output_dir, model_name, "_model.csv"))
#
#        # Write the model summary to a CSV file
#        write.csv(summary(model)$fixed,
#                paste0(output_dir, model_name, "_model.summary.csv"))
#
#        # Write the Bayes R2 to a CSV file
#        write.csv(as.data.frame(r2),
#                paste0(output_dir, model_name, "_bayes_R2.csv"))
#      }
#    }
#  }
#
#  return(models)
# }



#Previous function that I knew worked but the updated one (02/24/2025)required me to run
models for year and doy. THe updated one should run seperate ones for
  #each and also the interaction which cuts down on run time (hopefully)
#create_brm_models <- function(data_file, response_data, additional_predictors = NULL,
# iter = 2000, cores = 2, init = 0,
# warmup = 200, chains = 2,
# control = list(adapt_delta = 0.95)) {
#   #set seed
#   set.seed(888)
#   # Read the CSV file
```

```
#   data <- read.csv(data_file)
#   # Create a list to store the models
#   models <- list()
#   # Function to determine the family based on the response variable name
#   get_family <- function(response) {
#     if (grepl("^leaf.*area", response) || grepl("\\.freq$", response) ||  grepl("chewvalue",
response)) {
#       return("zero_inflated_beta")
#     } else if (grepl("\\.rich$", response)) {
#       return("zero_inflated_poisson")
#     } else {
#       stop(paste("Unknown family for response variable:", response))
#     }
#   }
#   # Get unique response variables
#   response_vars <- unique(data$Response)
#   # Loop through each response variable
#   for (response in response_vars) {
#     # Get predictor variables for the current response
#     predictors <- data %>%
#       filter(Response == response) %>%
#       pull(Variable)
#     # Combine with additional predictors if provided
#     all_predictors <- unique(c(predictors, additional_predictors))
#     # Loop through each predictor for the current response
#     for (predictor in all_predictors) {
#       # Create the formula. CHECK THE PREDICTORS. Should be either + scale(year.x),
scale(doy.clean), and their interaction depending on the model
#       formula_str <- paste(response, "~", paste0("scale(", predictor, ")"), " + scale(year.x)") #edit
this line when running old or young leaf models because  "+ leaf_rel_age",
"+leaf_rel_age*scale(doy.clean)" shouldn't be included in those. Young leave end up having a
high Rhat with both year and DOY so I'm now running the models with each individual variale.
#       # if (!is.null(additional_predictors)) {
#       #   formula_str <- paste(formula_str,
#       #                        "+ leaf_rel_age + year.x")
#       # } #uncomment if wanted later??
#       formula <- as.formula(formula_str)
#       # Determine the family
#       family <- get_family(response)
#       # Fit the model
```

```
#       model <- brm(formula, data = response_data, family = family,
#                    iter = iter, cores = cores, init = init, warmup = warmup,
#                    chains = chains, control = control)
#       # Calculate Bayes R2
#       r2 <- bayes_R2(model) #Added on 02/13/2025 to give us the R2 for each model
#       # Store the model
#       modelname <- paste(response, predictor, sep = "")
#       models[[model_name]] <- model
#       #Write full model output
#       write.csv(model, paste0("Script_output/Brms/old/scale(year)/", model_name,
"_model.csv")) #update these to write age specific names
#       # Write the model summary to a CSV file
#       write.csv(summary(model)$fixed,
#                 paste0("Script_output/Brms/old/scale(year)/", model_name, "_model.summary.csv"))
#here too
#       # Write the Bayes R2 to a CSV file
#       #DO NOT NEED TO USE STEP3D.1 anymore because of this line.
#       write.csv(as.data.frame(r2),
#                 paste0("Script_output/Brms/old/scale(year)/", model_name, "_bayes_R2.csv")) #here
too too (added 02/13/2025)
#     }
#   }
#   return(models)
# }

#Old simple function----
# #year and doy models only. No climate variables
# simp_brm_models <- function(data, iter = 2000, cores = 2, init = 0,
#                             warmup = 200, chains = 2,
#                             control = list(adapt_delta = 0.95)) {
#
#   # Set seed for reproducibility
#   set.seed(888)
#
#   # Create a list to store the models
#   models <- list()
#
#   # Get response variable names from the data
#   freq_vars <- grep("\\.freq$", names(data), value = TRUE)
#   rich_vars <- grep("\\.rich$", names(data), value = TRUE)
```

```
#   perc_vars <- grep("^leafperc_", names(data), value = TRUE)
#   chew_vars <- grep("chewvalue", names(data), value = TRUE)  # Added chew value
#
#   # Combine all response variables
#   response_vars <- c(freq_vars, rich_vars, perc_vars, chew_vars)
#
#   # Function to determine the family based on the response variable name
#   get_family <- function(response) {
#     if (grepl("\\.freq$", response) || grepl("^leafperc_", response) || grepl("chewvalue",
response)) {
#       return("beta")
#     } else if (grepl("\\.rich$", response)) {
#       return("zero_inflated_poisson")
#     }
#   }
#
#   # Loop through each response variable
#   for (response in response_vars) {
#     # Create formulas for different model combinations
#     formulas <- list(
#       year = paste(response, "~ scale(year.x)+ (1|catalogNumber)"),
#       doy = paste(response, "~ scale(doy.clean)+ (1|catalogNumber)"),
#       interaction = paste(response, "~ scale(year.x) * scale(doy.clean)+ (1|catalogNumber)")
#     )
#
#     # Loop through each formula
#     for (model_type in names(formulas)) {
#       formula <- as.formula(formulas[[model_type]])
#
#       # Determine the family
#       family <- get_family(response)
#
#       # Fit the model
#       model <- brm(formula, data = data, family = family,
#                    iter = iter, cores = cores, init = init,
#                    warmup = warmup, chains = chains, control = control)
#
#       # Calculate Bayes R2
#       r2 <- bayes_R2(model) #Added on 02/13/2025 to give us the R2 for each model
#
```

```
#       # Store the model
#       model_name <- paste(response, model_type, sep = "_")
#       models[[model_name]] <- model
#
#
#       # Write full model output
#       write.csv(model,
#               file = paste0("Script_output/Brms/simp_models/l.lvl/young/", model_name,
"_model.csv"))
#
#       # Write the model summary
#       write.csv(summary(model)$fixed,
#               file = paste0("Script_output/Brms/simp_models/l.lvl/young/", model_name,
"_model_summary.csv"))
#
#       #Bays R2 files
#       write.csv(as.data.frame(r2),
#               paste0("Script_output/Brms/simp_models/l.lvl/young/", model_name,
"_bayes_R2.csv")) #here too too (added 02/13/2025)
#     }
#   }
#
#   return(models)
# }

# #Subset data for simple models
# oldleaves4 <- oldleaves %>%
#   select(c(ends_with(".freq"), ends_with(".rich"), starts_with("leafperc_"),
"chewvalue","year.x", "doy.clean", "catalogNumber"))
#
# youngleaves4 <- youngleaves %>%
#   select(c(ends_with(".freq"), ends_with(".rich"), starts_with("leafperc_"),
"chewvalue","year.x", "doy.clean","catalogNumber"))
#
# # Usage of function----
# #simple year and doy models below:
# simp_oldmodels <- simp_brm_models(oldleaves4)
# simp_youngmodels <- simp_brm_models(youngleaves4)
```